
Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop v19.1.1

Performed by the OpenHack Security Agent

Issued by Tim Schnepf (security@openhack.com)

2026-02-20

Contents

1 Document Control	2
2 Executive Summary	3
3 Execution of the Security Penetration Test	4
3.1 Subject Under Test	4
3.2 Scope	4
3.3 Methodology	4
3.4 Events	5
4 Risk Assessment	6
4.1 CVSS	6
4.2 Risk Categories	7
4.3 Vulnerability States	8
5 Summary of Findings	9
6 Findings and Remediation	10
6.1 Finding	10
6.1.1 Application Configuration Exposure via /rest/admin/application-configuration	10
6.2 Finding	11
6.2.1 User Data Exposure via /rest/memories Endpoint	11
6.3 Finding	11
6.3.1 SQL Injection in Login Form - Authentication Bypass	11
6.4 Finding	12
6.4.1 SQL Injection in Product Search Endpoint	12
6.5 Finding	13
6.5.1 Mass Assignment Privilege Escalation	13
6.6 Finding	13
6.6.1 Password Hash Exposure via Memories API	13
6.7 Finding	14
6.7.1 IDOR/BOLA: Unauthorized Access to User Profiles	14
6.8 Finding	15
6.8.1 IDOR/BOLA: Unauthorized Access to Shopping Baskets	15

6.9	Finding	16
6.9.1	Multiple Reflected XSS Vulnerabilities in Search Functionality	16
6.10	Finding	17
6.10.1	CORS Misconfiguration - Wildcard Origin Allowed	17
6.11	Finding	17
6.11.1	XXE via XML File Upload in Complaint Form	17
7	Appendix A - Lists, Scripts and Raw Data	19
8	Appendix B - List of Abbreviations	20

DRAFT

Scope

OpenHack Security Agent was engaged by The OWASP Foundation, Inc. to conduct an external IT security investigation. The subject of the investigation was the “OWASP Juice Shop v19.1.1”.

The test was performed using a local instance of the application at `localhost:3333` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘The OWASP Foundation, Inc.’. This report has been prepared exclusively for ‘The OWASP Foundation, Inc.’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-20

Version History

Version	Date	State	Author	Comments
0.1	2026-02-20	Draft	Tim Schnepf	-
1.0	2026-02-20	Final document	Tim Schnepf	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec
Security Analyst	Business Logic Tester	555-0100	analyst@test.com
Security Analyst	Business Logic Tester	555-0100	analyst@test.com

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org
Juice Shop Admin	Application Owner	555-0200	admin@juiceshop.com

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by The OWASP Foundation, Inc. (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the OWASP Juice Shop v19.1.1 application took place on **2026-02-20**.

For this purpose, the web application, accessible at <http://localhost:3333>, was tested from the perspective of an external attacker.

As part of the test, a total of 11 vulnerabilities were identified: 6 critical, 5 high, 0 medium, 0 low, and 0 informational.

Critical: **6** | High: **5** | Medium: **0** | Low: **0** | Informational: **0** | Total: **11**

Table 2.1: Overview of Findings

Severity Count	
Critical	X
High	X
Medium	X
Low	X
Informational	X
Total	X

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the OWASP Juice Shop v19.1.1 took place on 2026-02-20.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v4.0 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into four groups: “Base”, “Threat”, “Environmental” and “Supplemental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Threat group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating. The Supplemental group provides additional context that does not modify the final score.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the CVSS-B (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Attack Requirements (AT)
- Privileges Required (PR)
- User Interaction (UI)
- Vulnerable System Impact: Confidentiality (VC), Integrity (VI), Availability (VA)
- Subsequent System Impact: Confidentiality (SC), Integrity (SI), Availability (SA)

As penetration testers, we can only assess the general threats posed by a vulnerability (CVSS-B). However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers the Environmental metrics (CVSS-BE). This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been submitted before they are remedied. This changes the assessment of the risk and thus

the urgency of remediation of a vulnerability. For more information, see [CVSS v4.0 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v4.0 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v4 Score
01	Application Configuration Exposure via /rest/admin/application-configuration	Critical	N/A
02	User Data Exposure via /rest/memories Endpoint	Critical	N/A
03	SQL Injection in Login Form - Authentication Bypass	Critical	9.8
04	SQL Injection in Product Search Endpoint	Critical	9.1
05	Mass Assignment Privilege Escalation	Critical	9.8
06	Password Hash Exposure via Memories API	Critical	9.1
07	IDOR/BOLA: Unauthorized Access to User Profiles	High	8.1
08	IDOR/BOLA: Unauthorized Access to Shopping Baskets	High	7.5
09	Multiple Reflected XSS Vulnerabilities in Search Functionality	High	6.1
10	CORS Misconfiguration - Wildcard Origin Allowed	High	7.5
11	XXE via XML File Upload in Complaint Form	High	7.5

6 Findings and Remediation

6.1 Finding

6.1.1 Application Configuration Exposure via /rest/admin/application-configuration

Severity	Critical
CVSS Base Score	N/A
Assets	N/A
Status	draft

Description

The endpoint /rest/admin/application-configuration is accessible without authentication and exposes the complete application configuration including: Google OAuth client ID (1005568560502-6hm16lef8oh46hr2d98vf2ohlj4nfhq.apps.googleusercontent.com), full product catalog with pricing and metadata, user reviews and associated usernames, security question/answer pairs for geo-stalking challenges, memory/caption data with user information, CTF configuration and challenge settings, internal URLs and redirect configurations. This represents a critical information disclosure vulnerability that exposes sensitive application configuration data, OAuth credentials, and user-related information without requiring authentication.

Proof of Concept

Request: GET /rest/admin/application-configuration | Response: Contains full application configuration including Google OAuth client_id, product data, user reviews, security answers, and more

Remediation

1. Implement proper authentication and authorization checks on administrative endpoints
2. Remove sensitive configuration data from API responses
3. Use environment variables for OAuth credentials
4. Implement principle of least privilege for API access

6.2 Finding

6.2.1 User Data Exposure via /rest/memories Endpoint

Severity	Critical
CVSS Base Score	N/A
Assets	N/A
Status	draft

Description

The /rest/memories endpoint is accessible without authentication and exposes comprehensive user data including: User ID numbers, usernames and email addresses, MD5 password hashes (crackable), user roles (admin, deluxe, customer), deluxe membership tokens, TOTP secrets, profile image paths, account creation dates. Complete user database exposure enables credential theft, privilege escalation attacks, account takeover, and provides attackers with valid usernames/emails for targeted phishing or brute force attacks. MD5 hashes can be cracked to reveal plaintext passwords.

Proof of Concept

Request: GET /rest/memories | Response: JSON array containing 10+ user records with email, password hashes (MD5), roles, and deluxe tokens | Sample: bkimminich (admin) - Email: bjoern.kimminich@gmail.com - Password hash: 6edd9d726cbdc873c539e41ae8757b8c

Remediation

1. Remove or protect the /rest/memories endpoint with proper authentication
2. Never expose password hashes through API endpoints
3. Implement proper access controls to ensure users can only access their own data
4. Use bcrypt or Argon2 for password hashing instead of MD5

6.3 Finding

6.3.1 SQL Injection in Login Form - Authentication Bypass

Severity	Critical
CVSS Base Score	9.8 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N)
Assets	N/A

Status	draft
--------	-------

Description

The login endpoint at POST /rest/user/login is vulnerable to SQL injection. By injecting the payload 'admin@juice-sh.op' OR '1'=1 into the email field, an attacker can bypass authentication and gain access as any user, including administrators. Full authentication bypass - attacker receives valid JWT token with admin privileges including email, password hash, and role.

Proof of Concept

Endpoint: POST /rest/user/login | Payload: {email: admin@juice-sh.op' OR '1'=1, password: password} | Impact: Full authentication bypass - receives JWT token with admin user data: {email: admin@juice-sh.op, role: admin, password: 0192023a7bbd7325516f069df18b500}

Remediation

Use parameterized queries/prepared statements for all database interactions. Never concatenate user input directly into SQL queries. Implement proper input validation and sanitization.

6.4 Finding

6.4.1 SQL Injection in Product Search Endpoint

Severity	Critical
CVSS Base Score	9.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:H/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The product search endpoint at GET /rest/products/search?q= is vulnerable to multiple types of SQL injection including error-based, boolean-based blind, and time-based blind SQL injection. The vulnerability exists because user input is directly concatenated into SQL queries without proper sanitization. Database confirmed: SQLite (confirmed via error messages). Boolean-based blind SQL injection confirmed - can extract database version, schema, and table contents bit by bit.

Proof of Concept

Endpoint: GET /rest/products/search | Parameter: q | Payloads: Error-based: apple' (returns SQLITE_ERROR: near '%: syntax error) | Boolean True: apple' AND 1=1 AND '1'='1 | Boolean False: apple' AND 1=2 AND '1'='1 | Time-based: apple' AND randblob(100000000) AND '1'='1

Remediation

Use parameterized queries for the search functionality. Implement proper input validation and escape special characters. Consider using an ORM or query builder that automatically handles parameterization.

6.5 Finding

6.5.1 Mass Assignment Privilege Escalation

Severity	Critical
CVSS Base Score	9.8 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The registration and user update endpoints accept arbitrary fields that allow privilege escalation. By including the role field in user creation/update requests, an attacker can create admin accounts or escalate existing accounts to administrative privileges.

Proof of Concept

POST /api/Users/ with body including role: admin creates a user with admin privileges. The API accepts and processes the role parameter without validation against the requesting user's permissions.

Remediation

Implement strict field filtering on mass assignment endpoints. Use allowlists instead of blocklists for acceptable fields. Never allow role or privilege fields to be set by end users.

6.6 Finding

6.6.1 Password Hash Exposure via Memories API

Severity	Critical
CVSS Base Score	9.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The /rest/memories endpoint exposes sensitive user information including MD5 password hashes, deluxe tokens, TOTP secrets, and full user profiles without authentication. This allows attackers to harvest credentials and authentication tokens for all users including admin accounts. Attackers can obtain password hashes for offline cracking, steal deluxe membership tokens for unauthorized premium access, and potentially impersonate any user including administrators.

Proof of Concept

Vulnerable URL: <http://localhost:3333/rest/memories> | Exposed data: User passwords (MD5), deluxe tokens, TOTP secrets, email addresses, roles | Sample exposed user: {email: bjoern.kimminich@gmail.com, password: 6edd9d726cbdc873c539e41ae8757b8c, role: admin, deluxeToken: ''}

Remediation

Implement authentication requirements for the memories endpoint, remove sensitive fields from API responses, and consider using bcrypt or Argon2 for password hashing instead of MD5.

6.7 Finding

6.7.1 IDOR/BOLA: Unauthorized Access to User Profiles

Severity	High
CVSS Base Score	8.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The application fails to validate object-level authorization on the `/api/Users/{id}` endpoint, allowing any authenticated user to access any other user's profile information, including administrators. This is a Broken Object Level Authorization (BOLA) vulnerability. An attacker can enumerate user IDs sequentially to harvest email addresses, roles, and other profile information of all system users.

Proof of Concept

As authenticated User 24 (`testuser1_idor@test.com`), successfully accessed: Admin User (ID 1): email: `admin@juice-sh.op`, role: admin | User 2: `jim@juice-sh.op` (customer) | User 3: `bender@juice-sh.op` (customer) | User 4: `bjoern.kimminich@gmail.com` (admin) | User 5: `ciso@juice-sh.op` (deluxe)

Remediation

Implement strict object-level authorization checks on all API endpoints that access user-specific resources:

1. Verify that the requesting user's ID matches the resource being accessed
2. Use middleware to validate ownership before processing GET requests
3. Implement proper access control lists (ACLs) for administrative functions
4. Consider using indirect reference maps (GUIDs instead of sequential IDs) to prevent enumeration
5. Log and monitor unauthorized access attempts

6.8 Finding

6.8.1 IDOR/BOLA: Unauthorized Access to Shopping Baskets

Severity	High
CVSS Base Score	7.5 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The `/rest/basket/{id}` endpoint lacks proper authorization checks, allowing any authenticated user to view the complete contents of any other user's shopping basket. This exposes purchase intent, product preferences, and potential financial information.

Proof of Concept

As authenticated User 24 (`testuser1_idor@test.com`), successfully accessed: Admin's Basket (Basket ID 1, User ID 1): Apple Juice (-5030 qty), Orange Juice (3 qty), Eggfruit Juice (1 qty),

Permafrost 2020 Edition (\$9999.99) | User 2's Basket (Basket ID 2, User ID 2): Raspberry Juice (2 qty) | Successfully enumerated baskets 1-7

Remediation

Implement proper authorization checks on basket endpoints: 1. Verify the authenticated user's ID matches the UserId associated with the basket 2. Use session-based basket identification rather than URL parameters 3. Implement middleware to check basket ownership before returning data 4. Consider using JWT claims to store the user's basket ID 5. Return 403 Forbidden for unauthorized basket access attempts

6.9 Finding

6.9.1 Multiple Reflected XSS Vulnerabilities in Search Functionality

Severity	High
CVSS Base Score	6.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:R/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The search functionality at `/#/search` contains multiple reflected Cross-Site Scripting (XSS) vulnerabilities. The application fails to properly sanitize user input from the 'q' query parameter before rendering it in the DOM, allowing attackers to inject and execute arbitrary JavaScript code. Successful exploitation allows attackers to: Execute arbitrary JavaScript in victim's browser, Steal session cookies and authentication tokens, Perform actions on behalf of the victim, Deface the application, Redirect users to malicious websites.

Proof of Concept

Payloads: `img onerror: | iframe srcdoc: alert('XSS-Iframe')>` | URL: `http://localhost:3333/#/search?q=`
| Evidence: Multiple JavaScript alert dialogs executed with custom XSS messages confirming arbitrary code execution

Remediation

1. Implement strict output encoding for all user-supplied input rendered in the DOM 2. Use Angular's built-in sanitization mechanisms (DomSanitizer) 3. Implement Content Security Policy (CSP) headers to mitigate XSS impact 4. Validate and sanitize the 'q' parameter on both client and server side 5. Consider using a whitelist approach for allowed HTML tags if rich content is needed

6.10 Finding

6.10.1 CORS Misconfiguration - Wildcard Origin Allowed

Severity	High
CVSS Base Score	7.5 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The OWASP Juice Shop API returns 'Access-Control-Allow-Origin: *' header on all endpoints, allowing any website to make cross-origin requests to the API. This overly permissive CORS configuration could allow attackers to exfiltrate data or perform actions on behalf of authenticated users from malicious websites.

Proof of Concept

All API endpoints tested returned CORS wildcard header, allowing cross-origin requests from any domain. Affected endpoints: /rest/user/login, /api/Users/, /api/BasketItems/, /api/Products/, /api/Challenges/, /rest/products/search, /rest/admin/application-configuration. Headers: access-control-allow-origin: *, access-control-allow-methods: GET,HEAD,PUT,PATCH,POST,DELETE

Remediation

Configure CORS to only allow specific trusted origins. Replace 'Access-Control-Allow-Origin: *' with explicit domain whitelist. Implement proper origin validation on the server side.

6.11 Finding

6.11.1 XXE via XML File Upload in Complaint Form

Severity	High
CVSS Base Score	7.5 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N)
Assets	N/A
Status	draft

Description

The complaint form at `/api/Complaints/` accepts XML B2B order files without proper validation, allowing XML External Entity (XXE) attacks. By uploading an XML file with an external entity declaration pointing to system files, an attacker can read arbitrary files from the server.

Proof of Concept

Attack Vectors: Upload malicious XML with `<!ENTITY xxe SYSTEM 'file:///etc/passwd'>` | XML parser processes external entities and exposes file contents | Evidence: Successfully solved 'XXE Data Access' and 'Deprecated Interface' challenges by uploading XML with external entity pointing to `/etc/passwd`

Remediation

Disable external entity processing in XML parser, use allowlist for expected XML structure, validate and sanitize all uploaded XML files

DRAFT

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+